

LLD Learning Series - Day 6

Topic: Design Patterns - Strategy Design Pattern

What are Design Patterns?

Design patterns are recurring solutions to common problems that developers face while building applications. They represent the collective wisdom of past developers who encountered and solved similar challenges.

The Core Idea Behind All Design Patterns

Every application has two kinds of parts: parts that stay the same (static) and parts that change frequently (dynamic). Good design means identifying the dynamic parts early, extracting them into separate classes, and isolating them from the stable parts. Each design pattern addresses this differently.

The Problem: Inheritance Gone Wrong

To understand why the Strategy Pattern exists, it helps to see what happens when we rely too heavily on inheritance.

Robot Simulation Example

Start with a Robot base class having walk(), talk(), and an abstract projection().

Subclasses CompanionRobot and WorkerRobot override projection() and inherit walk and talk.

A new SparrowRobot needs a fly() method. Add it to SparrowRobot. Then CrowRobot also needs fly(). Same code is copy-pasted.

To avoid duplication, fly() is moved to the Robot base class. Now all robots fly, which is wrong.

The next attempt introduces intermediate classes: FlyableRobot, NonFlyableRobot, then further splits for wing-flying vs jet-flying, walking vs non-walking, and so on.

The inheritance tree explodes. Adding one new robot type or behaviour requires touching multiple classes and creating new subclasses.

Why Inheritance Fails Here

Problem	What Goes Wrong
Code duplication	Same fly() logic repeated across multiple subclasses. Violates DRY.
Rigid hierarchy	Adding a new behaviour combination requires a new subclass. Combinatorial explosion.
OCP violation	Every new behaviour forces modification of existing class hierarchy.
Poor reuse	Shared behaviour (e.g. normal walking) cannot be reused without copying or restructuring.

The Strategy Design Pattern

Definition: Define a family of algorithms, put each of them into a separate class, and make their objects interchangeable so that the algorithm can be changed at runtime.

The Key Insight

Identify the parts of a class that change (the behaviours) and extract them into their own classes. The main class then holds references to these behaviour objects and delegates to them, instead of implementing the behaviour itself.

Applying Strategy to the Robot Example

Identify what varies: talk(), walk(), fly() change across robot types. projection() is stable.

Create separate interfaces for each varying behaviour:

- Talkable with a talk() method.

- Walkable with a walk() method.

- Flyable with a fly() method.

Create concrete implementations for each variant:

- Talk: NormalTalk, NoTalk

- Walk: NormalWalk, NoWalk

- Fly: NormalFly, NoFly

Robot holds references to Talkable, Walkable, and Flyable. These are injected via the constructor.

Robot delegates: robot.walk() calls walkable.walk(), robot.fly() calls flyable.fly().

Code Structure

```
// Behaviour interfaces
class Talkable { virtual void talk() = 0; };
class Walkable { virtual void walk() = 0; };
class Flyable { virtual void fly() = 0; };

// Concrete behaviours
class NormalTalk : public Talkable { void talk() { /* normal talk */ } };
class NoFly : public Flyable { void fly() { /* does nothing */ } };

// Robot uses composition
class Robot {
    Talkable* talkable;
    Walkable* walkable;
    Flyable* flyable;
    Robot(Talkable* t, Walkable* w, Flyable* f) { ... }
    void talk() { talkable->talk(); }
    void walk() { walkable->walk(); }
    void fly() { flyable->fly(); }
    virtual void projection() = 0;
};
```

Creating Robot Instances

CompanionRobot: pass NormalTalk, NormalWalk, NoFly.

SparrowRobot: pass NormalTalk, NormalWalk, NormalFly.

Any future robot type is just a new combination of existing strategies. No existing class changes.

Behaviours can even be swapped at runtime by replacing the strategy reference on the Robot object.

UML Structure of the Strategy Pattern

Component	Role	In the Robot Example
Client	The class that holds and uses a strategy	Robot class
Strategy	Abstract interface defining the common method	Talkable, Walkable, Flyable
Concrete Strategy	Specific implementation of the strategy interface	NormalTalk, NoFly, NormalWalk, etc.

The Client has a has-a relationship to Strategy and invokes behaviour through the interface, never caring about the concrete type behind it.

Real-World Examples

Payment System

A Payment class holds a reference to a PaymentStrategy interface.

Concrete strategies: UPI, CreditCard, NetBanking.

Calling payNow() executes whichever strategy is currently assigned. Switching payment method requires no change to Payment class.

Sorting Algorithms

A Sorter class holds a reference to a SortStrategy interface.

Concrete strategies: QuickSort, MergeSort, InsertionSort.

The sorting algorithm can be swapped at runtime based on data size or type, with no change to the Sorter class.

Key Concepts at a Glance

Concept	What it Means
Favor composition over inheritance	Hold references to behaviour objects instead of subclassing to get behaviour.
DRY (Don't Repeat Yourself)	A shared behaviour like NormalFly is written once and reused across all robots that need it.
Open/Closed Principle	Add a new behaviour by creating a new strategy class. Nothing existing is modified.
Runtime swapping	Strategies can be replaced on a live object, making behaviour truly dynamic.
Delegation	The Robot class is a dumb shell. It calls the strategy, not implements the logic itself.

My Takeaways from Today

1. Design patterns are not new syntax or libraries. They are proven ways of structuring code that solve recurring design problems. Learning them means learning how experienced developers think.
2. The inheritance explosion problem with robots made it very clear why deep hierarchies are fragile. Every new combination of behaviour needed a new subclass. That is not scalable.
3. Strategy solves this by asking: what changes? Extract that. What stays the same? Keep that. Once I identified talk, walk, and fly as the dynamic parts, the rest followed naturally.
4. Composition over inheritance is now something I understand at a gut level, not just as a phrase. Robot does not IS-A talker. It HAS-A talker. That distinction changes how the design works.

5. The payment and sorting examples showed that Strategy is everywhere. Anytime you have a family of algorithms that need to be swappable, that is a Strategy pattern waiting to be applied.

6. The Robot becoming a dumb object that just delegates is elegant. The logic lives in the strategy, not in the class that uses it. That is clean separation of concerns.